CS532 Quiz 8: 3D Meshes

Name: _____

Manifold vs. non-manifold failure case: You are given a polygonal mesh that visually looks like a closed watertight object, but when you try to build a half-edge data structure, your code crashes on a few "problem" edges.

- **Q1.1** Discuss how the manifold conditions on edges and vertex fans may relate to (or cause) this failure.

  Answer:

  A half-edge structure assumes a 2-manifold mesh locally:

  Edge condition: Each undirected edge is incident to at most two faces (boundary: 1 face, interior: 2 faces). If an edge is singular (3+ incident faces), there is no way to assign exactly two half-edges with consistent left/right faces, so your half-edge builder will either overwrite pointers or fail.

  Vertex fan condition: Around each vertex, the incident faces must form a single open or closed fan; if the neighborhood contains branching or multiple disconnected fans, you cannot traverse it as a simple loop with next/prev pointers.

  Orientability: Half-edges also rely on a consistent notion of "left" and "right" faces. That requires that adjacent faces have compatible orientation (shared edge appears with opposite vertex order in the two faces). If one face is flipped relative to its neighbors, the attempt to assign left/right faces along the edge will conflict.

Mesh Representations: Consider three ways to represent a mesh: (a) independent faces ("triangle soup"), (b) vertex and face tables with indices, (c) a full half-edge structure.

For each of the following tasks, discuss which of these representations would better facilitate the following tasks

- **Q2.1** GPU-friendly rendering of a static mesh
- **Q2.2** Repeated edge collapses for mesh simplification
- **Q3.3** Fast queries of all faces adjacent to given vertex v.

Answers:
Q2.1 GPU-friendly rendering of a static mesh
Best: Vertex and face tables (b).

- You want compact vertex buffer with index buffer; faces are just triples of indices into a shared vertex array.
- Triangle soup (a) repeats coordinates, wasting memory and bandwidth and giving no sharing.
- Half-edge (c) adds lots of pointer-like adjacency info that the GPU doesn't use for simple rasterization

Q2.2 Repeated edge collapses for mesh simplification
Best: Half-edge (c).

- You need to:
  - Find the two faces adjacent to the edge,
  - Update neighboring edges/vertices,
  - Remove faces and update connectivity locally.
- Half-edge stores all adjacencies in O(1) time with fixed records, making local edits like collapse/subdivide cheap and robust.
- Triangle soup or plain vertex/face tables would require expensive global scans or extra bookkeeping for every collapse.

Q2.3 Fast query: "all faces adjacent to this vertex"
Best: Half-edge (c) (or winged edge), with adjacency lists as an alternative.

- In half-edge, each vertex stores a pointer $v.inc_edge$ to one outgoing half-edge; by following twin + next around the vertex, you enumerate all incident faces in time proportional to their number.
- Adjacency lists explicitly store all vertex–face adjacencies, which also gives O(degree) traversal but at higher storage cost.
- Triangle soup / bare vertex–face tables have no built-in adjacency; you'd need to build a separate map or scan all faces.

So conceptually: vertex/face tables are great for one-shot rendering, but half-edge is better when you need repeated, local topological queries and edits.